Building data course — Week 3 — fault detection in SQL

sparql-playground / lessons

Generated for print and offline reading

Week 3 — fault detection in SQL

Five daily lessons. Run the SQL in DB Browser for SQLite against the API's own database, `var/timeseries.sqlite` (build it with `uv run bldgts bootstrap`). Lines like `-- expect: 288` are the answer you should see. Keep notes under the ignored `student-work/` folder.

Contents

- Day 11 — Long to wide: the `history` table every Open-FDD rule reads
- Day 12 — Indexes and `EXPLAIN QUERY PLAN`

- Day 13 — Window functions: LAG, rolling averages, streaks
- Day 14 — Write a fault rule (FC1, duct static too low) and check it against Open-FDD
- Day 15 — What production does differently (Timescale), and the UUID bridge to RDF

Day 11 — Long to wide: the `history` table every Open-FDD rule reads

Goal: pivot the readings into one row per AHU per timestamp, one column per role.

Read

Day 8 needed six joins to put three points side by side. Fault rules want *many* points side by side, so do the pivot once:

```
long (samples + lab_point)
wide (lab_history)
equipment ts     role          value
equipment ts     fan_status duct_static
duct_static_sp ...
AHU_1     10:05  fan-status    1
—▶     AHU_1     10:05  1          1.31
1.40
AHU_1     10:05  duct-static   1.31
AHU_1     10:05  duct-static-sp 1.40
```

The SQL trick is **conditional aggregation** again (Day 9): group by `(equipment_id, ts)`, and for each column pick the value of one role.

```
MAX(CASE WHEN openfdd_role = 'duct-static-
pressure' THEN value END) AS duct_static
```

Each group has exactly one reading per role, so `MAX` simply picks it. The other rows give NULL, which `MAX` ignores.

Open-FDD's SQL rules (see `open-fdd/sql_rules/*.sql`) all read a relation called `history` with columns named after roles: `fan_status`, `fan_cmd`, `duct_static`, `duct_static_sp`, `sat`, `mat`, `rat`, `oat`… The column names below follow that **input contract**, so a rule written against it doesn't care which building it runs on.

You'll make it a **table**, not a view: the pivot reads 924k rows and takes a few seconds. As a table it's built once and every rule after that is fast (the Day 10 summary-table idea).

Run

```
DROP TABLE IF EXISTS lab_history;
CREATE TABLE lab_history AS
SELECT p.equipment_id, s.ts,
  MAX(CASE WHEN p.openfdd_role = 'fan-
         status'              THEN
         s.value END) AS fan_status,
  MAX(CASE WHEN p.openfdd_role = 'fan-cmd'
         THEN s.value END) AS fan_cmd,
  MAX(CASE WHEN p.openfdd_role = 'duct-
         static-pressure'    THEN s.value
         END) AS duct_static,
  MAX(CASE WHEN p.openfdd_role = 'duct-
         static-pressure-sp' THEN s.value
         END) AS duct_static_sp,
  MAX(CASE WHEN p.openfdd_role =
         'discharge-air-temp'      THEN
         s.value END) AS sat,
  MAX(CASE WHEN p.openfdd_role =
         'discharge-air-temp-sp'   THEN
         s.value END) AS sat_sp,
  MAX(CASE WHEN p.openfdd_role = 'mixed-
         air-temp'          THEN s.value
         END) AS mat,
  MAX(CASE WHEN p.openfdd_role = 'return-
         air-temp'         THEN s.value
         END) AS rat,
  MAX(CASE WHEN p.openfdd_role = 'outside-
         air-temp'        THEN s.value
         END) AS oat,
  MAX(CASE WHEN p.openfdd_role = 'outside-
         air-damper'      THEN s.value
         END) AS oa_damper,
  MAX(CASE WHEN p.openfdd_role = 'cooling-
         valve'           THEN s.value
         END) AS clg_valve,
  MAX(CASE WHEN p.openfdd_role =
         'occupied'                THEN
         s.value END) AS occupied
FROM samples s
```

```
JOIN lab_point p ON p.timeseries_id =
         s.timeseries_id
WHERE p.openfdd_role IS NOT NULL
GROUP BY p.equipment_id, s.ts;

SELECT count(*) FROM lab_history;
-- expect: 71072
```

2 AHUs × 35,536 timestamps. Browse it in DB Browser: it looks like the original CSV again, but with standard column names.

```
-- Day 8's six-join query, now one line of
         logic
SELECT equipment_id,
         round(avg(duct_static), 3),
         round(avg(duct_static_sp), 3),
         count(*)
FROM lab_history
WHERE fan_status = 1
GROUP BY equipment_id
ORDER BY equipment_id;
-- expect: AHU_1 | 1.347 | 1.402 | 18893
```

Try it

1. Data quality: how many rows have the fan running but a discharge, mixed or return air temperature of exactly 0?
2. What's missing from this pivot if a role has **no** point on some AHU? (Think about what the column would hold.)

Answers

```
SELECT count(*) FROM lab_history
WHERE fan_status = 1 AND (sat = 0 OR mat =
          0 OR rat = 0);
-- expect: 14
```

Fourteen dropout rows while running: rare, but each one could trip a rule ("discharge air 50 °F below setpoint!"). Production rules filter or flag them.

2. The column would be NULL for that AHU. That's correct: "not measured" is not 0. A rule that needs that role must then **skip** the AHU and say so, rather than invent data. Open-FDD's contract calls that "skipped / not applicable".

Why it matters

Which UUID becomes `duct_static` for which AHU is a **metadata** question. Here `lab_point.openfdd_role` answered it. In Unit 4 a SPARQL query will: "for each AHU, the point with role duct-static-pressure and its `ref:hasTimeseriesId`". That's the model-driven way to build this exact table for any building.

Day 12 — Indexes and `EXPLAIN QUERY PLAN`

Goal: see how SQLite finds rows, and add an index where it has to read everything.

Read

A table without a useful index is a list: to find rows, SQLite **scans** all of them, O(n). An **index** is a B-tree sorted by some columns. Finding a key or a key range is O(log n), then it walks the matching rows in order.

`samples` is `WITHOUT ROWID` with primary key `(timeseries_id, ts)`, so the table **is** a B-tree sorted by series, then time. That makes some questions cheap and others not:

Query filters on	Can use the key?
`timeseries_id = ?`	yes, it's the first key column
`timeseries_id = ? AND ts BETWEEN ...`	yes, both columns, range on the second
`ts BETWEEN ...` alone	**no**: time is only sorted *within* each series, so it must scan
`date(ts, 'unixepoch') = ...`	the function hides `ts` from the index

Prefix `EXPLAIN QUERY PLAN` to any query and SQLite tells you its plan instead of running it. Look for `SEARCH ... USING PRIMARY KEY` / `USING INDEX` (good) vs `SCAN table` (reads everything).

The tradeoff: every index speeds up some reads, but costs disk space and slows every insert. Historians index exactly what they're queried by (series + time) and little else.

Run

```
EXPLAIN QUERY PLAN
SELECT * FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
          9069-3b0f17d49c48'
  AND ts >= unixepoch('2026-07-01') AND ts
          < unixepoch('2026-07-02');
```

`SEARCH samples USING PRIMARY KEY (timeseries_id=? AND ts>? AND ts<?)` jumps straight to 288 rows.

```
EXPLAIN QUERY PLAN
SELECT * FROM samples
WHERE ts >= unixepoch('2026-07-01') AND ts
          < unixepoch('2026-07-02');
```

`SCAN samples`: all 4.2 million rows, to return 33,984. Run both queries without `EXPLAIN` and compare the time DB Browser reports under the results.

```
-- lab_history has no index yet, so even
        one AHU-day scans
EXPLAIN QUERY PLAN
SELECT * FROM lab_history
WHERE equipment_id = 'AHU_1'
  AND ts >= unixepoch('2026-07-01') AND ts
        < unixepoch('2026-07-02');


CREATE INDEX IF NOT EXISTS
        lab_history_equipment_ts ON
        lab_history (equipment_id, ts);

SELECT count(*) FROM lab_history
WHERE equipment_id = 'AHU_1'
  AND ts >= unixepoch('2026-07-01') AND ts
        < unixepoch('2026-07-02');
-- expect: 288
```

Run the `EXPLAIN QUERY PLAN` again: `SEARCH lab_history USING INDEX lab_history_equipment_ts`.

Try it

1. Check the plan for Day 3's "bad" filter, `date(ts, 'unixepoch') = '2026-07-01'`, with a `timeseries_id`. Which part of the key does it still use?
2. If dashboards often ask "all points at one moment" (`ts = ?` across every series), what index would help, and what would it cost?

Answers

```
EXPLAIN QUERY PLAN
SELECT * FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
        9069-3b0f17d49c48'
  AND date(ts, 'unixepoch') = '2026-07-
        01';
```

```
SEARCH samples USING PRIMARY KEY
(timeseries_id=?)
```

. It finds the series fast, then checks the date on all 35,536 of its rows instead of 288.

2. An index on `samples(ts)` (or `(ts, timeseries_id)`). It would roughly duplicate the key data, around another 100+ MB here, and slow down every insert. Timescale solves the “by time” access differently: it splits the table into time **chunks** (Day 15).

Why it matters

The same thinking applies to triple stores (week 5). A graph of `(subject, predicate, object)` triples needs several sorted indexes (SPO, POS, OSP) because SPARQL queries fix different positions. Which index a query can use is the whole performance story.

Day 13 — Window functions: LAG, rolling averages, streaks

Goal: compare each row with its neighbours in time, without collapsing rows like `GROUP BY` does.

Read

A **window function** computes a value for each row from a "window" of related rows. All rows stay in the result.

```
LAG(x)        OVER (PARTITION BY
equipment_id ORDER BY ts)   -- x from the
previous row
LEAD(x)       OVER (...)
-- x from the next row
ROW_NUMBER()  OVER (PARTITION BY ... ORDER
BY ts)             -- 1, 2, 3 ... within
each partition
AVG(x)        OVER (PARTITION BY ... ORDER
BY ts
                    ROWS BETWEEN 11
PRECEDING AND CURRENT ROW) -- rolling mean
of the last 12 rows (1 hour)
SUM(x)        OVER (PARTITION BY ... ORDER
BY ts)             -- running total
```

- `PARTITION BY equipment_id` keeps AHU_1 and AHU_2 separate. Without it, AHU_2's first row would "follow" AHU_1's last.
- `ORDER BY ts` defines what "previous" means.

Streaks (gaps and islands). Fault rules need "this condition has been true for N rows in a row". The standard trick takes three steps:

1. Mark where the value changes: `CASE WHEN x = LAG(x) OVER w THEN 0 ELSE 1 END AS new_streak`.
2. A running sum of those marks numbers each streak: `SUM(new_streak) OVER w AS streak_id`.
3. Number the rows inside each streak: `ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id ORDER BY ts)`.

A window function can't be used directly in `WHERE`, because windows are computed after filtering. Wrap the query in a subquery, or a `WITH name AS (...)` common table expression (CTE), and filter outside.

Run

```
-- Each reading next to the previous one
SELECT datetime(ts, 'unixepoch') AS t,
         duct_static,
       LAG(duct_static) OVER (PARTITION BY
         equipment_id ORDER BY ts) AS
         prev
FROM lab_history
WHERE equipment_id = 'AHU_1'
ORDER BY ts
LIMIT 3;
-- expect rows: 3
```

The first row's `prev` is NULL: nothing came before it.

```
-- Fan starts: off in the previous row, on
         in this one
WITH w AS (
  SELECT equipment_id, ts, fan_status,
         LAG(fan_status) OVER (PARTITION
         BY equipment_id ORDER BY ts) AS
         prev
  FROM lab_history
)
SELECT equipment_id, count(*) AS starts
FROM w
WHERE fan_status = 1 AND prev = 0
GROUP BY equipment_id
ORDER BY equipment_id;
-- expect: AHU_1 | 123
```

About one start a day for 123 days, as you'd expect for a scheduled AHU.

```sql
-- Longest continuous run of AHU_1's fan,
        in hours
WITH marks AS (
  SELECT ts, fan_status,
         CASE WHEN fan_status =
         LAG(fan_status) OVER (ORDER BY
         ts) THEN 0 ELSE 1 END AS
         new_streak
  FROM lab_history WHERE equipment_id =
         'AHU_1'
), streaks AS (
  SELECT *, SUM(new_streak) OVER (ORDER BY
         ts) AS streak_id FROM marks
)
SELECT round(max(n) * 5 / 60.0, 1) AS
         longest_run_h
FROM (SELECT streak_id, count(*) AS n FROM
         streaks WHERE fan_status = 1
         GROUP BY streak_id);
-- expect: 109.4
```

109 hours: the fan ran for more than four days straight. For an office AHU that's worth asking about (an override? a schedule left in occupied?). It's the same question Day 9's unoccupied runtime raised.

Try it

1. A 1-hour rolling average of AHU_2's duct static pressure. What's its maximum?
2. Count AHU_1 fan **stops** (on, then off). Should it equal the starts? Why might it differ by one?

Answers

```
SELECT round(max(r), 3) FROM (
  SELECT AVG(duct_static) OVER (PARTITION
         BY equipment_id ORDER BY ts
                                ROWS
         BETWEEN 11 PRECEDING AND CURRENT
         ROW) AS r
  FROM lab_history WHERE equipment_id =
         'AHU_2'
);
-- expect: 1.675


WITH w AS (
  SELECT fan_status, LAG(fan_status) OVER
         (ORDER BY ts) AS prev
  FROM lab_history WHERE equipment_id =
         'AHU_1'
)
SELECT count(*) FROM w WHERE fan_status =
         0 AND prev = 1;
-- expect: 122
```

122 stops against 123 starts. They can differ by one because the data can begin or end in the middle of a run (it ends with the fan running).

Why it matters

“Condition true for N consecutive rows” is how a fault rule ignores a one-sample blip and reports only persistent problems. Tomorrow that’s the confirmation step of a real rule.

Day 14 — Write a fault rule (FC1, duct static too low) and check it against Open-FDD

Goal: turn an engineering rule into SQL, count fault hours, and compare with a production FDD engine's results.

Read

Fault detection and diagnostics (FDD) runs rules over trend data to find equipment that isn't doing what it should. Open-FDD's rules come from ASHRAE Guideline 36 and the NIST/PNNL APAR rules. **FC1** says:

> The supply fan is running (at or near full speed) but duct static pressure stays below its setpoint.

The fan is working as hard as it can and still can't make pressure. Likely causes: a broken belt, a failing VFD, a blocked filter or coil, a leaky duct, or a setpoint too high for this system.

Every rule in `open-fdd/sql_rules/` follows the same shape (read `fc1_duct_static_low.sql` after this lesson):

1. **Gate**: only judge rows where the equipment is on (`fan_status = 1`). Off-hours aren't faults.
2. **Raw fault**: `CASE WHEN <condition> THEN 1 ELSE 0 END`, with a tolerance so noise doesn't trip it.
3. **Confirm**: count a row only once the raw fault has held for N consecutive rows (Day 13's streaks).
4. **Aggregate**: fault rows × 5 minutes = fault hours.

Our simplified FC1 uses `duct_static < duct_static_sp - 0.1` (inH2O), gates on `fan_status = 1`, and confirms after 2 rows (10 minutes). Open-FDD's real FC1 has more knobs (fan speed, mode-change delay, coverage); `sql/fdd/` documents them.

Run

```
DROP TABLE IF EXISTS lab_fc1;
CREATE TABLE lab_fc1 AS
WITH base AS (
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND
         duct_static < duct_static_sp -
         0.1 THEN 1 ELSE 0 END AS
         raw_fault
  FROM lab_history
), marks AS (
  SELECT *, CASE WHEN raw_fault =
         LAG(raw_fault) OVER (PARTITION
         BY equipment_id ORDER BY ts)
                 THEN 0 ELSE 1 END AS
         new_streak
  FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER
         (PARTITION BY equipment_id ORDER
         BY ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION
         BY equipment_id, streak_id ORDER
         BY ts) AS streak_len FROM
         streaks
)
SELECT equipment_id, ts, raw_fault,
         streak_id,
       CASE WHEN raw_fault = 1 AND
         streak_len >= 2 THEN 1 ELSE 0
         END AS fault
FROM ranked;

SELECT equipment_id,
       round(sum(raw_fault) * 5 / 60.0, 1)
         AS raw_fault_h,
       round(sum(fault) * 5 / 60.0, 1)
         AS confirmed_fault_h,
```

```
       count(DISTINCT CASE WHEN fault = 1
         THEN streak_id END) AS episodes
FROM lab_fc1
GROUP BY equipment_id
ORDER BY equipment_id;
-- expect: AHU_1 | 120 | 114.3 | 53
```

AHU_1 has about 114 confirmed fault hours in 53 episodes. AHU_2 has 1.5 hours: essentially healthy for this rule (its problem is the opposite: static *above* setpoint, Day 8).

Compare with Open-FDD. Its results for this data ship in `data/BUILDING_50/*/fdd_events.csv`. Load them:

```
uv run python lessons/course.py load-
         events
```

That creates `lab_fdd_event` (one row per fault episode: `equipment_id`, `rule_name`, `start_ts`, `end_ts`, `duration_min`).

```
SELECT rule_name, equipment_id, count(*)
         AS episodes,
         round(sum(duration_min) / 60, 1)
         AS hours
FROM lab_fdd_event
WHERE rule_name = 'fc1'
GROUP BY rule_name, equipment_id;
-- expect: fc1 | AHU_1 | 80 | 484.1
```

80 episodes and 484 hours, against our 53 and 114. Different numbers don't mean either is wrong yet. Check whether the two engines are finding **the same events**:

```
-- How many of Open-FDD's FC1 episodes
         contain at least one of our
         confirmed fault rows?
SELECT count(*) AS openfdd_episodes,
       sum(EXISTS (SELECT 1 FROM lab_fc1 f
                   WHERE f.equipment_id =
         e.equipment_id AND f.fault = 1
                     AND f.ts BETWEEN
         e.start_ts AND e.end_ts)) AS
         also_found_by_us
FROM lab_fdd_event e
WHERE e.rule_name = 'fc1';
-- expect: 80 | 73
```

73 of 80 overlap: same problems, measured differently. Look at the episode lengths (`max(duration_min)` is about 1,425 minutes, nearly a day). Open-FDD's episode clock runs from first to last fault sample, including stretches in between, and its event timestamps aren't on this 5-minute grid. It ran on its own copy of the data with its own tuning. Totals depend on those choices. Overlap tells you whether the rules agree on **what** is wrong.

Try it

1. Change the tolerance from 0.1 to 0.2 inH2O. How much do AHU_1's confirmed hours drop?
2. Write the opposite rule, **duct static too high** (`duct_static > duct_static_sp + 0.1` while running), and run it on AHU_2.

Answers

```
WITH base AS (
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND
         duct_static < duct_static_sp -
         0.2 THEN 1 ELSE 0 END AS
         raw_fault
  FROM lab_history WHERE equipment_id =
         'AHU_1'
), marks AS (
  SELECT *, CASE WHEN raw_fault =
         LAG(raw_fault) OVER (ORDER BY
         ts) THEN 0 ELSE 1 END AS
         new_streak FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (ORDER BY
         ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION
         BY streak_id ORDER BY ts) AS
         streak_len FROM streaks
)
SELECT round(sum(raw_fault = 1 AND
         streak_len >= 2) * 5 / 60.0, 1)
         FROM ranked;
-- expect: 101.3
```

From 114.3 to 101.3 hours. Most of AHU_1's shortfall is more than 0.2 inH2O: a real problem, not a marginal one.

```
WITH base AS (
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND
         duct_static > duct_static_sp +
         0.1 THEN 1 ELSE 0 END AS
         raw_fault
  FROM lab_history WHERE equipment_id =
         'AHU_2'
), marks AS (
  SELECT *, CASE WHEN raw_fault =
         LAG(raw_fault) OVER (ORDER BY
         ts) THEN 0 ELSE 1 END AS
         new_streak FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (ORDER BY
         ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION
         BY streak_id ORDER BY ts) AS
         streak_len FROM streaks
)
SELECT round(sum(raw_fault = 1 AND
         streak_len >= 2) * 5 / 60.0, 1)
         AS high_static_h FROM ranked;
-- expect: 1428.3
```

About 1,428 hours, nearly all of AHU_2's 1,500 run hours. The fan pushes far more pressure than its setpoint asks for, all the time: wasted fan energy, and possibly a reset sequence that isn't working. Open-FDD's version of this rule is `ahu_ducthi.sql`.

Why it matters

Notice what the rule needed from outside the data: which column is “duct static” and which is “fan status” for each AHU. Change buildings and only that mapping changes. That’s the case for a semantic model. The rule stays the same, and the model tells it where the inputs are.

Day 15 — What production does differently (Timescale), and the UUID bridge to RDF

Goal: know which parts of Unit 1 carry over to a production historian, and see where the RDF model picks up.

Read

SQLite here, Timescale in production

Open-FDD's production stack keeps history in **PostgreSQL + TimescaleDB**. Everything you wrote this unit is plain SQL and carries over. The differences are operational:

Need	This course (SQLite)	Timescal
Store readings	`samples(timeseries_id, ts, value)`, key on series + time	a **hypertabl** same colum split into tim **chunks** automaticall so "last week only touches few chunks

Need	This course (SQLite)	Timescal
Bucket by time	`ts / 3600 * 3600`	`time_bucket hour', time`
Summary tables	`CREATE TABLE lab_hourly AS ...`, rebuilt by hand	**continuou aggregates** refreshed incrementall data arrives
Old data	keep everything	**compressi** and **retenti policies** pe chunk
Many writers and readers	one writer at a time	a server with concurrent connections roles
Wide `history` input	`lab_history` table	a view or temporary ta built for one site, then the rule SQL run as is

The rules don’t change. The Open-FDD SQL you read yesterday runs on Postgres with small dialect changes (`sql/fdd/README.md` lists them). SQLite needs a few more, like sample standard deviation, which it lacks.

The UUID bridge

All unit long you used `lab_point` to know that `52422584-…` is AHU_1's duct static pressure. The API doesn't use a table for that. It uses the RDF model, `model/building_50.ttl`. Open it in a text editor and search for `52422584-9894-50bd-9696-2fbb8b062518`:

```
bldg:AHU_1_DA_P a bts:Entity ;
    rdfs:label "AHU1 DA-P" ;
    ref:hasExternalReference [ a
ref:TimeseriesReference ;
            ref:hasTimeseriesId "52422584-
9894-50bd-9696-2fbb8b062518" ;
            ref:storedAt
bldg:timeseries_db ] ;
    bts:hasHaystackMarker "air",
"discharge", "his", ...
```

Read it as statements, each one **subject, predicate, object**:

- `AHU_1_DA_P` has label "AHU1 DA-P".
- `AHU_1_DA_P` has an external reference, a TimeseriesReference with id `52422584-…`, stored at `bldg:timeseries_db`.
- `AHU_1_DA_P` has the Haystack markers `air`, `discharge`, …

And further down, `bldg:timeseries_db` describes the database you've been querying, including its schema: `samples(timeseries_id TEXT, ts INTEGER epoch-s UTC, value REAL)`.

So the model is a graph of statements about things, and one kind of statement points into the time-series store by UUID. Units 2–4 teach you to store, query and build graphs like this. Then the question “give me duct static and fan status for every AHU” is answered by the model. The answer comes back as UUIDs, and those go into the SQL you already know.

Run

```
-- What the course added to the API's
        database
SELECT type, name FROM sqlite_master WHERE
        name LIKE 'lab_%' ORDER BY type,
        name;


-- The one-line summary of Unit 1:
        confirmed FC1 hours per AHU per
        month
SELECT equipment_id, strftime('%Y-%m', ts,
        'unixepoch') AS month,
       round(sum(fault) * 5 / 60.0, 1) AS
        fc1_hours
FROM lab_fc1
GROUP BY equipment_id, month
ORDER BY equipment_id, month;
-- expect: AHU_1 | 2026-03
```

When you’re done, `uv run python lessons/course.py reset` drops every `lab_*` object. The API’s data is untouched.

Unit 1 challenge

Write a **discharge air temperature** rule for AHU_1 and AHU_2: while the fan runs, discharge air temperature (`sat`) is more than 5 °F away from its setpoint (`sat_sp`) for at least 30 minutes. Then:

1. Exclude dropout rows (`sat = 0`) and explain how many rows that removed.
2. Report fault hours per AHU per month.
3. Pick the worst day and chart `sat`, `sat_sp` and `fan_status` for it in DB Browser's **Plot** tab.
4. Write two sentences an operator could act on.

Next: **Unit 2** builds a tiny triple store in Python, so the Turtle above stops being mysterious.

Reading — Week 3

The lessons first, then these for exact behavior.

- SQLite window functions
- SQLite query planner
- SQLite EXPLAIN QUERY PLAN
- Open-FDD SQL cookbook
- TimescaleDB hypertables
- TimescaleDB continuous aggregates